



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ Информатика, искусственный интеллект и системы управления

КАФЕДРА Системы обработки информации и управления

ОТЧЕТ

по домашнему заданию

по дисциплине «**МЕТОДЫ МАШИННОГО ОБУЧЕНИЯ**»

Тема: «Сравнительный анализ методов обучения с подкреплением для задач управления агентом в средах Taxi-v4 и CartPole-v1»

Студент	<u>ИУ5И-22М</u>	<u></u>	<u>Тавуз Мохамад</u>
	(группа)	(подпись, дата)	(И.О. Фамилия)
Преподаватель		<u></u>	<u>Ю.Е.Гапанюк</u>
		(подпись, дата)	(И.О. Фамилия)

2026 г.

1. Введение

Обучение с подкреплением является одним из важных направлений современного машинного обучения и используется для решения задач, в которых необходимо обучить агента принимать последовательные решения на основе взаимодействия с окружающей средой. В отличие от классического обучения с учителем, где модель обучается на заранее размеченных данных, в обучении с подкреплением агент самостоятельно получает опыт, выполняя действия, наблюдая изменения состояния среды и получая вознаграждение за свои действия.

Актуальность выбранной темы связана с тем, что методы обучения с подкреплением применяются в задачах робототехники, автоматического управления, игровых системах, логистике, рекомендательных системах и других областях, где требуется построение оптимальной стратегии поведения. При этом существуют как классические табличные методы, применимые к дискретным средам, так и методы глубокого обучения с подкреплением, позволяющие работать с более сложными пространствами состояний.

В данной работе рассматривается сравнительный анализ нескольких методов обучения с подкреплением. В качестве классических табличных алгоритмов исследуются Policy Iteration, SARSA, Q-Learning и Double Q-Learning. В качестве метода глубокого обучения с подкреплением рассматривается алгоритм Deep Q-Network. Эксперименты выполняются в средах Taxi-v4 и CartPole-v1 библиотеки Gymnasium. По результатам экспериментов проводится сравнение алгоритмов по времени обучения, средней награде, количеству шагов и успешности выполнения задачи.

2. Постановка выбранной задачи машинного обучения

В качестве задачи машинного обучения в данной работе выбрана задача обучения с подкреплением, связанная с обучением агента принимать решения в дискретных и непрерывных средах управления. Обучение с подкреплением является направлением машинного обучения, в котором агент взаимодействует со средой, получает информацию о текущем состоянии, выбирает действие, получает вознаграждение и переходит в новое

состояние. Цель агента состоит в формировании такой стратегии поведения, которая обеспечивает максимизацию суммарного ожидаемого вознаграждения.

В практической части работы рассматриваются две среды из библиотеки Gymnasium: Taxi-v4 и CartPole-v1. Среда Taxi-v4 используется для исследования табличных методов обучения с подкреплением, так как она имеет конечное дискретное пространство состояний и действий. В этой среде агент должен научиться управлять такси, забрать пассажира из заданной точки и доставить его в пункт назначения. Среда CartPole-v1 используется для реализации алгоритма Deep Q-Network, поскольку состояние среды задается непрерывным вектором признаков, а для аппроксимации Q-функции применяется нейронная сеть.

Целью домашнего задания является сравнительный анализ классических табличных методов обучения с подкреплением и метода глубокого обучения с подкреплением. Для этого в работе реализуются и исследуются алгоритмы Policy Iteration, SARSA, Q-Learning, Double Q-Learning и Deep Q-Network. Качество работы алгоритмов оценивается по средней суммарной награде, среднему количеству шагов, доле успешных эпизодов и времени обучения.

3. Теоретическая часть отчета

3.1. Основные понятия обучения с подкреплением

Обучение с подкреплением представляет собой подход к машинному обучению, при котором обучаемый объект, называемый агентом, взаимодействует со средой и постепенно формирует стратегию поведения на основе получаемого опыта. В каждый момент времени агент находится в некотором состоянии среды, выбирает одно из допустимых действий, после чего среда возвращает новое состояние и численное вознаграждение. На основе этой информации агент изменяет свое поведение таким образом, чтобы в дальнейшем получать большую суммарную награду.

Основными элементами задачи обучения с подкреплением являются агент, среда, состояние, действие, вознаграждение и стратегия. Агент представляет собой обучаемую

систему, принимающую решения. Среда описывает внешнюю систему, с которой взаимодействует агент. Состояние содержит информацию о текущей ситуации, действие определяет выбор агента, а вознаграждение используется как численная оценка качества выбранного действия. Стратегия определяет правило выбора действий агентом в различных состояниях.

Формально многие задачи обучения с подкреплением описываются как марковский процесс принятия решений. В такой постановке предполагается, что будущее состояние среды зависит от текущего состояния и выбранного действия, а не от всей предыдущей истории взаимодействия. Цель обучения состоит в нахождении оптимальной стратегии, максимизирующей ожидаемую суммарную дисконтированную награду. Для этого используются функции ценности состояния $V(s)$ и функции ценности действия $Q(s, a)$, которые позволяют оценивать полезность состояний и действий.

3.2. Табличные методы обучения с подкреплением

Табличные методы обучения с подкреплением применяются в задачах, где пространство состояний и пространство действий являются конечными и имеют сравнительно небольшой размер. В таких условиях значения функции $Q(s, a)$ могут храниться в виде таблицы, строки которой соответствуют состояниям, а столбцы — возможным действиям. Обновление значений в таблице выполняется на основе опыта взаимодействия агента со средой.

Одним из классических методов является Policy Iteration. Данный алгоритм относится к методам динамического программирования и предполагает, что модель среды известна, то есть известны вероятности переходов между состояниями и соответствующие вознаграждения. Алгоритм состоит из двух этапов: оценивания текущей стратегии и улучшения стратегии. На этапе оценивания вычисляется функция ценности для текущей стратегии, а на этапе улучшения для каждого состояния выбирается действие, обеспечивающее наибольшее ожидаемое значение. Процесс повторяется до тех пор, пока стратегия не перестает изменяться.

Методы SARSA и Q-Learning относятся к методам обучения на основе временных различий. Они не требуют полного знания модели среды и обновляют Q-значения непосредственно в процессе взаимодействия агента со средой. Алгоритм SARSA является on-policy методом, так как обновление выполняется с учетом действия, реально выбранного агентом в следующем состоянии. Алгоритм Q-Learning является off-policy методом, поскольку при обновлении используется максимальное Q-значение следующего состояния независимо от того, какое действие фактически будет выбрано агентом.

Double Q-Learning является модификацией Q-Learning, направленной на уменьшение переоценки Q-значений. В классическом Q-Learning одно и то же значение используется и для выбора лучшего действия, и для оценки этого действия, что может приводить к завышенной оценке качества действий. В Double Q-Learning используются две независимые Q-таблицы, которые обновляются попеременно. Такой подход позволяет сделать оценку более устойчивой.

3.3. Алгоритм Deep Q-Network

Алгоритм Deep Q-Network является развитием классического Q-Learning и применяется в тех случаях, когда хранение Q-функции в табличном виде становится невозможным или неэффективным. Основная идея DQN состоит в том, что функция $Q(s, a)$ аппроксимируется нейронной сетью. На вход сети подается состояние среды, а на выходе формируются Q-значения для всех возможных действий. После этого агент выбирает действие с максимальным Q-значением либо выполняет случайное действие в соответствии с epsilon-greedy стратегией.

Важной особенностью DQN является использование буфера воспроизведения опыта Replay Buffer. Во время взаимодействия со средой агент сохраняет переходы вида «состояние — действие — награда — следующее состояние — признак завершения эпизода». При обучении нейронной сети из буфера случайным образом выбираются мини-пакеты переходов. Это позволяет уменьшить зависимость между соседними наблюдениями и сделать процесс обучения более устойчивым.

Еще одним важным элементом DQN является целевая нейронная сеть Target Network. В алгоритме используются две сети: основная сеть, параметры которой обновляются на каждом шаге обучения, и целевая сеть, применяемая для вычисления целевых Q-значений. Периодическое обновление параметров целевой сети позволяет стабилизировать обучение и уменьшить колебания значений функции потерь.

В данной работе DQN применяется для среды CartPole-v1. Состояние среды описывается четырьмя числовыми признаками: положением тележки, скоростью тележки, углом наклона шеста и угловой скоростью. Пространство действий содержит два действия: перемещение тележки влево или вправо. Для аппроксимации Q-функции используется полносвязная нейронная сеть с двумя скрытыми слоями.

3.4. Анализ научных источников по выбранной задаче

В рамках теоретического этапа были рассмотрены научные источники, посвященные классическим и глубоким методам обучения с подкреплением. В качестве основных источников были использованы работа Watkins и Dayan, посвященная алгоритму Q-Learning, работа van Hasselt по алгоритму Double Q-Learning, а также статья Mnih и соавторов, в которой был предложен алгоритм Deep Q-Network. Эти источники позволяют рассмотреть развитие методов обучения с подкреплением от табличных алгоритмов к глубоким нейросетевым подходам.

Алгоритм Q-Learning является одним из наиболее известных off-policy методов обучения с подкреплением. Его особенность состоит в том, что агент обновляет Q-значения с использованием максимального значения функции Q для следующего состояния. Такой подход позволяет обучать оптимальную стратегию независимо от текущей стратегии поведения агента, применяемой для исследования среды. В то же время классический Q-Learning может приводить к завышению оценок Q-значений, что стало причиной появления модификации Double Q-Learning.

Double Q-Learning использует две независимые оценки функции Q и разделяет выбор действия и оценивание его качества. Это позволяет уменьшить эффект переоценки и

повысить устойчивость обучения. Данный подход особенно важен в задачах, где из-за случайности среды или ограниченного числа обучающих эпизодов оценки действий могут быть нестабильными.

Для задач с большими или непрерывными пространствами состояний используются методы глубокого обучения с подкреплением. Одним из ключевых алгоритмов является Deep Q-Network. В DQN вместо таблицы Q-значений применяется нейронная сеть, которая аппроксимирует функцию $Q(s, a)$. Важными элементами этого алгоритма являются Replay Buffer и Target Network. Replay Buffer используется для хранения и случайной выборки опыта, а Target Network применяется для стабилизации вычисления целевых Q-значений. Эти идеи позволяют применять Q-Learning к более сложным задачам, где табличное представление функции ценности невозможно.

4. Практическая часть отчета

4.1. Подготовка среды выполнения

Практический этап работы был выполнен в среде Google Colab с использованием языка программирования Python. Для реализации экспериментов применялись библиотеки Gymnasium, NumPy, Pandas, Matplotlib, tqdm и PyTorch. Библиотека Gymnasium использовалась для создания сред обучения с подкреплением Taxi-v4 и CartPole-v1. Библиотеки NumPy и Pandas применялись для численных вычислений и формирования итоговых таблиц. Matplotlib использовалась для визуализации процесса обучения, а PyTorch — для построения и обучения нейронной сети в алгоритме DQN.

На первом этапе была выполнена установка и импорт необходимых библиотек, а также проверена доступность вычислительного устройства. В результате проверки было установлено, что для выполнения экспериментов доступно устройство CUDA, что позволяет использовать графический ускоритель при обучении нейронной сети. Также были созданы папки results и figures для сохранения таблиц и графиков, полученных в ходе практических экспериментов.

```
▶ # Подготовка библиотек, проверка устройства и создание папок для результатов

!pip -q install gymnasium[toy-text] numpy pandas matplotlib tqdm torch torchvision

import os
import time
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tqdm import tqdm

import gymnasium as gym

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F

# Фиксация random seed для воспроизводимости экспериментов
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

# Выбор устройства для обучения нейронных сетей
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Создание папок для сохранения результатов
os.makedirs("results", exist_ok=True)
os.makedirs("figures", exist_ok=True)

print("Библиотеки успешно импортированы")
print("Версия Gymnasium:", gym.__version__)
print("Версия NumPy:", np.__version__)
print("Версия Pandas:", pd.__version__)
print("Версия PyTorch:", torch.__version__)
print("Используемое устройство:", device)

... 12.8/12.8 MB 56.0 MB/s eta 0:00:00
Библиотеки успешно импортированы
Версия Gymnasium: 1.3.0
Версия NumPy: 2.0.2
Версия Pandas: 2.2.2
Версия PyTorch: 2.11.0+cu128
Используемое устройство: cuda
```

Рисунок 1 — Импорт библиотек и проверка среды выполнения

4.2. Создание и исследование среды Taxi-v4

Для исследования табличных алгоритмов обучения с подкреплением была выбрана среда Taxi-v4 из библиотеки Gymnasium. Данная среда имеет дискретное пространство состояний и дискретное пространство действий, поэтому она подходит для реализации алгоритмов Policy Iteration, SARSA, Q-Learning и Double Q-Learning.

В среде Taxi-v4 агент управляет такси, которое должно забрать пассажира из одной точки и доставить его в заданный пункт назначения. По результатам анализа среды было определено, что пространство состояний содержит 500 возможных состояний, а пространство действий содержит 6 возможных действий. Также была рассмотрена структура переходов для выбранной пары «состояние–действие», что подтвердило возможность использования модели переходов среды в алгоритме Policy Iteration.

```
▶ # Создание среды Taxi-v4 и вывод ее основных характеристик

env = gym.make("Taxi-v4")
state, info = env.reset(seed=SEED)

n_states = env.observation_space.n
n_actions = env.action_space.n

print("Среда:", "Taxi-v4")
print("Пространство состояний:", env.observation_space)
print("Количество состояний:", n_states)
print("Пространство действий:", env.action_space)
print("Количество действий:", n_actions)
print("Начальное состояние:", state)

sample_state = 54
sample_action = 0

print("\nПример структуры переходов")
print("Состояние:", sample_state)
print("Действие:", sample_action)
print("Переходы:")

for transition in env.unwrapped.P[sample_state][sample_action]:
    probability, next_state, reward, terminated = transition
    print(
        "probability =", probability,
        "| next_state =", next_state,
        "| reward =", reward,
        "| terminated =", terminated
    )
```

Рисунок 2 — Код создания среды Taxi-v4 и вывода ее характеристик

```
... Среда: Taxi-v4
    Пространство состояний: Discrete(500)
    Количество состояний: 500
    Пространство действий: Discrete(6)
    Количество действий: 6
    Начальное состояние: 386

    Пример структуры переходов
    Состояние: 54
    Действие: 0
    Переходы:
    probability = 1.0 | next_state = 154 | reward = -1 | terminated = False
```

Рисунок 3 — Результат создания среды Taxi-v4 и вывода ее характеристик

4.3. Реализация и обучение алгоритма Policy Iteration

На следующем этапе был реализован алгоритм Policy Iteration для среды Taxi-v4. Данный алгоритм относится к методам динамического программирования и использует известную модель переходов среды. Реализация включает два основных этапа: оценивание текущей стратегии и улучшение стратегии.

На этапе оценивания стратегии для каждого состояния вычислялось значение функции ценности с учетом текущей стратегии, вероятностей переходов, вознаграждений и коэффициента дисконтирования. На этапе улучшения стратегии для каждого состояния вычислялись значения возможных действий, после чего выбиралось действие с максимальным ожидаемым значением. Процесс повторялся до стабилизации стратегии.

В результате обучения алгоритм Policy Iteration завершил работу за 17 итераций. Время обучения составило приблизительно 1,62 секунды. Полученная стратегия далее была использована для оценки качества работы агента на тестовых эпизодах.

```

▶ # Реализация алгоритма Policy Iteration для среды Taxi-v4

class PolicyIterationAgent:
    def __init__(self, environment, gamma=0.99, theta=1e-8, max_iterations=1000):
        self.env = environment
        self.n_states = environment.observation_space.n
        self.n_actions = environment.action_space.n
        self.gamma = gamma
        self.theta = theta
        self.max_iterations = max_iterations

        self.policy = np.zeros(self.n_states, dtype=int)
        self.value_function = np.zeros(self.n_states)

    def policy_evaluation(self):
        for _ in range(self.max_iterations):
            delta = 0

            for state in range(self.n_states):
                old_value = self.value_function[state]
                action = self.policy[state]

                new_value = 0
                for probability, next_state, reward, terminated in self.env.unwrapped.P[state][action]:
                    new_value += probability * (
                        reward + self.gamma * self.value_function[next_state] * (not terminated)
                    )

                self.value_function[state] = new_value
                delta = max(delta, abs(old_value - new_value))

            if delta < self.theta:
                break

    def policy_improvement(self):
        policy_stable = True

        for state in range(self.n_states):
            old_action = self.policy[state]
            q_values = np.zeros(self.n_actions)

            for action in range(self.n_actions):
                for probability, next_state, reward, terminated in self.env.unwrapped.P[state][action]:
                    q_values[action] += probability * (
                        reward + self.gamma * self.value_function[next_state] * (not terminated)
                    )

            best_action = np.argmax(q_values)
            self.policy[state] = best_action

            if old_action != best_action:
                policy_stable = False

        return policy_stable

    def train(self):
        start_time = time.time()

        for iteration in range(1, self.max_iterations + 1):
            self.policy_evaluation()
            policy_stable = self.policy_improvement()

            if policy_stable:
                break

        training_time = time.time() - start_time

        return {
            "Алгоритм": "Policy Iteration",
            "Количество итераций": iteration,
            "Время обучения, сек": training_time
        }

policy_iteration_agent = PolicyIterationAgent(env)
policy_iteration_training_results = policy_iteration_agent.train()

print("Результаты обучения Policy Iteration")
for key, value in policy_iteration_training_results.items():
    print(key, ":", value)

```

Рисунок 4 — Код реализации алгоритма Policy Iteration

```
... Результаты обучения Policy Iteration
Алгоритм : Policy Iteration
Количество итераций : 17
Время обучения, сек : 1.6183295249938965
```

Рисунок 5 — Результат обучения алгоритма Policy Iteration

4.4. Оценка качества алгоритма Policy Iteration

После обучения стратегии, полученной с помощью алгоритма Policy Iteration, была выполнена ее проверка на 100 тестовых эпизодах. Для оценки качества использовались три показателя: средняя суммарная награда, среднее количество шагов за эпизод и доля успешных эпизодов.

По результатам тестирования средняя награда составила 7,34, среднее количество шагов — 13,66, а доля успешных эпизодов — 1,0. Это означает, что обученная стратегия успешно завершила все тестовые эпизоды. Полученный результат показывает, что Policy Iteration эффективно решает задачу в среде Taxi-v4 при наличии известной модели переходов.

```
▶ # Оценка качества стратегии, полученной алгоритмом Policy Iteration

policy_iteration_eval_results = evaluate_policy(
    environment=env,
    policy=policy_iteration_agent.policy,
    episodes=100,
    max_steps=200,
    seed=SEED
)

print("Оценка качества Policy Iteration")
for key, value in policy_iteration_eval_results.items():
    print(key, ":", value)
```

```
... Оценка качества Policy Iteration
Средняя награда : 7.34
Среднее количество шагов : 13.66
Доля успешных эпизодов : 1.0
```

Рисунок 6 — Оценка качества стратегии Policy Iteration

4.5. Реализация алгоритмов SARSA, Q-Learning и Double Q-Learning

Далее были реализованы алгоритмы SARSA, Q-Learning и Double Q-Learning, относящиеся к методам обучения на основе временных различий. В отличие от Policy Iteration, данные алгоритмы не требуют полного знания модели переходов среды и обучаются непосредственно на основе опыта взаимодействия агента со средой.

Для всех трех алгоритмов был реализован базовый класс агента, содержащий Q-таблицу, параметры обучения и epsilon-greedy стратегию выбора действий. Q-таблица имеет размер, соответствующий количеству состояний и действий среды Taxi-v4. При выборе действия агент либо выполняет случайное действие для исследования среды, либо выбирает действие с максимальным текущим Q-значением.

В алгоритме SARSA обновление Q-значений выполняется с учетом действия, выбранного агентом в следующем состоянии. В алгоритме Q-Learning для обновления используется максимальное Q-значение следующего состояния. В алгоритме Double Q-Learning применяются две Q-таблицы, что позволяет уменьшить переоценку значений действий и повысить устойчивость обучения.

Реализация базового класса агента и алгоритмов SARSA, Q-Learning, Double Q-Learning

```
class BaseTDAgent:
    def __init__(
        self,
        environment,
        alpha=0.1,
        gamma=0.99,
        epsilon=1.0,
        epsilon_min=0.05,
        epsilon_decay=0.9995,
        episodes=8000,
        max_steps=200
    ):
        self.env = environment
        self.n_states = environment.observation_space.n
        self.n_actions = environment.action_space.n

        self.alpha = alpha
        self.gamma = gamma
        self.epsilon = epsilon
        self.epsilon_min = epsilon_min
        self.epsilon_decay = epsilon_decay
        self.episodes = episodes
        self.max_steps = max_steps

        self.Q = np.zeros((self.n_states, self.n_actions))
        self.episode_rewards = []
        self.episode_steps = []

    def choose_action(self, state):
        if np.random.random() < self.epsilon:
            return self.env.action_space.sample()
        return int(np.argmax(self.Q[state]))

    def decay_epsilon(self):
        self.epsilon = max(self.epsilon_min, self.epsilon * self.epsilon_decay)

    def get_policy(self):
        return np.argmax(self.Q, axis=1)

class SARSAAgent(BaseTDAgent):
    def train(self):
        start_time = time.time()

        for episode in tqdm(range(self.episodes), desc="SARSA"):
            state = reset_env(self.env, seed=SEED + episode)
            action = self.choose_action(state)

            total_reward = 0
            steps = 0
            done = False

            while not done and steps < self.max_steps:
                next_state, reward, done, info = step_env(self.env, action)
                next_action = self.choose_action(next_state)

                target = reward + self.gamma * self.Q[next_state, next_action] * (not done)
                self.Q[state, action] += self.alpha * (target - self.Q[state, action])

                state = next_state
                action = next_action
                total_reward += reward
                steps += 1

            self.episode_rewards.append(total_reward)
            self.episode_steps.append(steps)
            self.decay_epsilon()

        training_time = time.time() - start_time

        return {
            "Алгоритм": "SARSA",
            "Количество эпизодов": self.episodes,
            "Время обучения, сек": training_time,
            "Финальное epsilon": self.epsilon
        }

class QLearningAgent(BaseTDAgent):
```



```

class QLearningAgent(BaseTDAgent):
    def train(self):
        start_time = time.time()

        for episode in tqdm(range(self.episodes), desc="Q-Learning"):
            state = reset_env(self.env, seed=SEED + episode)

            total_reward = 0
            steps = 0
            done = False

            while not done and steps < self.max_steps:
                action = self.choose_action(state)
                next_state, reward, done, info = step_env(self.env, action)

                target = reward + self.gamma * np.max(self.Q[next_state]) * (not done)
                self.Q[state, action] += self.alpha * (target - self.Q[state, action])

                state = next_state
                total_reward += reward
                steps += 1

            self.episode_rewards.append(total_reward)
            self.episode_steps.append(steps)
            self.decay_epsilon()

        training_time = time.time() - start_time

        return {
            "Алгоритм": "Q-Learning",
            "Количество эпизодов": self.episodes,
            "Время обучения, сек": training_time,
            "Финальное epsilon": self.epsilon
        }

class DoubleQLearningAgent(BaseTDAgent):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.Q1 = np.zeros((self.n_states, self.n_actions))
        self.Q2 = np.zeros((self.n_states, self.n_actions))

    def choose_action(self, state):
        if np.random.random() < self.epsilon:
            return self.env.action_space.sample()
        return int(np.argmax(self.Q1[state] + self.Q2[state]))

    def get_policy(self):
        return np.argmax(self.Q1 + self.Q2, axis=1)

    def train(self):
        start_time = time.time()

        for episode in tqdm(range(self.episodes), desc="Double Q-Learning"):
            state = reset_env(self.env, seed=SEED + episode)

            total_reward = 0
            steps = 0
            done = False

            while not done and steps < self.max_steps:
                action = self.choose_action(state)
                next_state, reward, done, info = step_env(self.env, action)

                if np.random.random() < 0.5:
                    best_next_action = np.argmax(self.Q1[next_state])
                    target = reward + self.gamma * self.Q2[next_state, best_next_action] * (not done)
                    self.Q1[state, action] += self.alpha * (target - self.Q1[state, action])
                else:
                    best_next_action = np.argmax(self.Q2[next_state])
                    target = reward + self.gamma * self.Q1[next_state, best_next_action] * (not done)
                    self.Q2[state, action] += self.alpha * (target - self.Q2[state, action])

                state = next_state
                total_reward += reward
                steps += 1

            self.episode_rewards.append(total_reward)
            self.episode_steps.append(steps)
            self.decay_epsilon()

        self.Q = self.Q1 + self.Q2
        training_time = time.time() - start_time

        return {
            "Алгоритм": "Double Q-Learning",
            "Количество эпизодов": self.episodes,
            "Время обучения, сек": training_time,
            "Финальное epsilon": self.epsilon
        }

```

Рисунок 7 — Код реализации алгоритмов SARSA, Q-Learning и Double Q-Learning

4.6. Обучение и оценка алгоритмов SARSA, Q-Learning и Double Q-Learning

После реализации алгоритмов SARSA, Q-Learning и Double Q-Learning было выполнено их обучение в среде Taxi-v4. Для всех алгоритмов использовалось одинаковое количество обучающих эпизодов, равное 8000. В процессе обучения применялась epsilon-greedy стратегия, при которой начальное значение epsilon постепенно уменьшалось до минимального значения 0,05. Такой подход позволяет в начале обучения активно исследовать среду, а затем постепенно переходить к использованию найденной стратегии.

По результатам обучения было установлено, что все три алгоритма успешно завершили процесс обучения и достигли финального значения epsilon, равного 0,05. Время обучения алгоритма SARSA составило приблизительно 10,42 секунды, Q-Learning — 10,80 секунды, а Double Q-Learning — 12,59 секунды. Наибольшее время обучения показал Double Q-Learning, что связано с использованием двух Q-таблиц и дополнительными операциями при обновлении оценок.

После обучения была выполнена оценка качества полученных стратегий на 100 тестовых эпизодах. Все три алгоритма достигли доли успешных эпизодов 1,0. Средняя награда составила 7,30 для SARSA, 7,34 для Q-Learning и 7,34 для Double Q-Learning. Среднее количество шагов составило 13,70 для SARSA, 13,66 для Q-Learning и 13,66 для Double Q-Learning. Таким образом, Q-Learning и Double Q-Learning показали одинаковое качество на тестовых эпизодах, а SARSA продемонстрировал немного меньшую среднюю награду.

▶ # Обучение агентов SARSA, Q-Learning и Double Q-Learning на среде Taxi-v4

```
sarsa_agent = SARSAAgent(env)
q_learning_agent = QLearningAgent(env)
double_q_learning_agent = DoubleQLearningAgent(env)

sarsa_training_results = sarsa_agent.train()
q_learning_training_results = q_learning_agent.train()
double_q_learning_training_results = double_q_learning_agent.train()

training_results_df = pd.DataFrame([
    sarsa_training_results,
    q_learning_training_results,
    double_q_learning_training_results
])

display(training_results_df)
```

```
... SARSA: 100%|██████████| 8000/8000 [00:10<00:00, 768.19it/s]
Q-Learning: 100%|██████████| 8000/8000 [00:10<00:00, 740.76it/s]
Double Q-Learning: 100%|██████████| 8000/8000 [00:12<00:00, 635.61it/s]
```

	Алгоритм	Количество эпизодов	Время обучения, сек	Финальное epsilon
0	SARSA	8000	10.419275	0.05
1	Q-Learning	8000	10.804644	0.05
2	Double Q-Learning	8000	12.591701	0.05

Рисунок 8 — Результаты обучения SARSA, Q-Learning и Double Q-Learning

▶ # Оценка качества стратегий SARSA, Q-Learning и Double Q-Learning

```
sarsa_eval_results = evaluate_policy(  
    environment=env,  
    policy=sarsa_agent.get_policy(),  
    episodes=100,  
    max_steps=200,  
    seed=SEED  
)  
  
q_learning_eval_results = evaluate_policy(  
    environment=env,  
    policy=q_learning_agent.get_policy(),  
    episodes=100,  
    max_steps=200,  
    seed=SEED  
)  
  
double_q_learning_eval_results = evaluate_policy(  
    environment=env,  
    policy=double_q_learning_agent.get_policy(),  
    episodes=100,  
    max_steps=200,  
    seed=SEED  
)  
  
td_eval_results_df = pd.DataFrame([  
    {"Алгоритм": "SARSA", **sarsa_eval_results},  
    {"Алгоритм": "Q-Learning", **q_learning_eval_results},  
    {"Алгоритм": "Double Q-Learning", **double_q_learning_eval_results}  
)  
  
display(td_eval_results_df)
```

...	Алгоритм	Средняя награда	Среднее количество шагов	Доля успешных эпизодов
0	SARSA	7.30	13.70	1.0
1	Q-Learning	7.34	13.66	1.0
2	Double Q-Learning	7.34	13.66	1.0

Рисунок 9 — Оценка качества SARSA, Q-Learning и Double Q-Learning

4.7. Визуализация процесса обучения табличных алгоритмов

Для более подробного анализа процесса обучения были построены графики изменения суммарной награды и количества шагов по эпизодам для алгоритмов SARSA, Q-Learning и Double Q-Learning. Поскольку значения награды и количества шагов могут заметно изменяться от эпизода к эпизоду, для визуализации использовалось скользящее среднее с окном 100 эпизодов.

График изменения награды показывает, что на начальном этапе обучения значения суммарной награды являются низкими, так как агент активно исследует среду и часто выполняет неоптимальные действия. По мере накопления опыта Q-таблицы уточняются, и средняя награда постепенно увеличивается. Наиболее быстрое улучшение в начале обучения показал алгоритм Q-Learning, однако к концу обучения все три алгоритма достигли близких значений награды.

График изменения количества шагов показывает, что в начале обучения агенту требуется больше действий для завершения эпизода. После обучения количество шагов уменьшается, что говорит о формировании более эффективной стратегии. Алгоритмы Q-Learning и Double Q-Learning в конце обучения демонстрируют практически одинаковое поведение, а SARSA также достигает близкого результата.

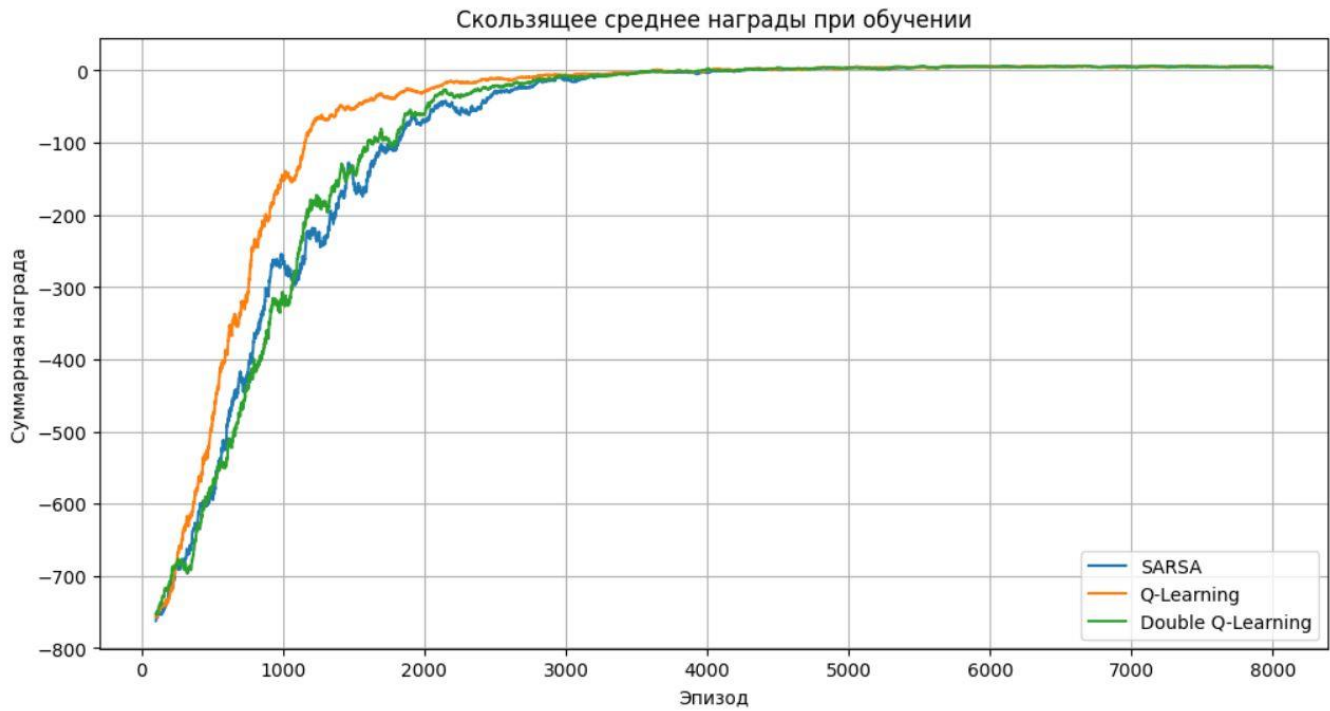


Рисунок 10 — График изменения награды при обучении SARSA, Q-Learning и Double Q-Learning



Рисунок 11 — График изменения количества шагов при обучении SARSA, Q-Learning и Double Q-Learning

4.8. Итоговое сравнение табличных алгоритмов

После обучения и тестирования всех табличных алгоритмов была сформирована итоговая таблица сравнения. В таблицу были включены результаты алгоритмов Policy Iteration, SARSA, Q-Learning и Double Q-Learning. Для сравнения использовались количество обучающих эпизодов или итераций, время обучения, средняя награда, среднее количество шагов и доля успешных эпизодов.

По результатам сравнения все рассмотренные алгоритмы смогли успешно решить задачу Taxi-v4 и достигли доли успешных эпизодов 1,0. Алгоритм Policy Iteration показал высокое качество стратегии и быстро завершил обучение, однако его применение возможно только при наличии известной модели переходов среды.

Среди методов временных различий наилучшие результаты по средней награде показали Q-Learning и Double Q-Learning. Их средняя награда составила 7,34, а среднее количество шагов — 13,66. Алгоритм SARSA также успешно решил задачу, но его средняя награда была немного ниже и составила 7,30, а среднее количество шагов — 13,70. Время обучения Double Q-Learning оказалось больше, чем у SARSA и Q-Learning, что объясняется использованием двух Q-таблиц.

	Алгоритм	Количество эпизодов	Количество итераций	Время обучения, сек	Средняя награда	Среднее количество шагов	Доля успешных эпизодов
0	Policy Iteration	-	17	1.618330	7.34	13.66	1.0
1	SARSA	8000	-	10.419275	7.30	13.70	1.0
2	Q-Learning	8000	-	10.804644	7.34	13.66	1.0
3	Double Q-Learning	8000	-	12.591701	7.34	13.66	1.0

Рисунок 12 — Итоговая таблица сравнения табличных алгоритмов

4.9. Создание и исследование среды CartPole-v1

Для исследования метода глубокого обучения с подкреплением была выбрана среда CartPole-v1 из библиотеки Gymnasium. Данная среда отличается от Taxi-v4 тем, что пространство наблюдений является непрерывным. Поэтому хранение функции $Q(s, a)$ в виде обычной таблицы является нецелесообразным, и для оценки Q -значений используется нейронная сеть.

В среде CartPole-v1 агент управляет тележкой с закрепленным на ней шестом. Цель агента состоит в том, чтобы перемещать тележку влево или вправо и удерживать шест в вертикальном положении как можно дольше. Состояние среды задается четырьмя числовыми признаками: положением тележки, скоростью тележки, углом наклона шеста и угловой скоростью. Пространство действий содержит два возможных действия.

В ходе выполнения эксперимента было определено, что количество признаков состояния равно 4, а количество действий равно 2. Эти характеристики позволяют использовать компактную полносвязную нейронную сеть для аппроксимации Q -функции в алгоритме DQN.

▶ # Создание среды CartPole-v1 и вывод ее основных характеристик

```
cartpole_env = gym.make("CartPole-v1")
state, info = cartpole_env.reset(seed=SEED)

cartpole_n_observations = cartpole_env.observation_space.shape[0]
cartpole_n_actions = cartpole_env.action_space.n

print("Среда:", "CartPole-v1")
print("Пространство наблюдений:", cartpole_env.observation_space)
print("Количество признаков состояния:", cartpole_n_observations)
print("Пространство действий:", cartpole_env.action_space)
print("Количество действий:", cartpole_n_actions)
print("Начальное состояние:")
print(state)
```

```
*** Среда: CartPole-v1
Пространство наблюдений: Box([-4.8          -inf -0.41887903      -inf], [4.8          inf 0.41887903      inf], (4,), float32)
Количество признаков состояния: 4
Пространство действий: Discrete(2)
Количество действий: 2
Начальное состояние:
[ 0.0273956 -0.00611216  0.03585979  0.0197368 ]
```

Рисунок 13 — Создание среды CartPole-v1 и вывод ее характеристик

4.10. Реализация Replay Buffer и нейронной сети DQN

Для реализации алгоритма Deep Q-Network были созданы два основных компонента: буфер воспроизведения опыта Replay Buffer и нейронная сеть DQN. Replay Buffer используется для хранения переходов, полученных агентом при взаимодействии со средой. Каждый переход содержит состояние, выбранное действие, полученную награду, следующее состояние и признак завершения эпизода.

Использование Replay Buffer позволяет обучать нейронную сеть не только на последних наблюдениях, но и на случайных мини-пакетах опыта, накопленного ранее. Это уменьшает зависимость между последовательными состояниями и делает процесс обучения более устойчивым.

Нейронная сеть DQN была реализована как полносвязная сеть. На вход сети подается вектор состояния размерности 4, соответствующий признакам среды CartPole-v1. На выходе формируются два Q-значения, соответствующие двум возможным действиям агента. Архитектура сети включает два скрытых слоя по 128 нейронов с функцией активации ReLU. Такая архитектура является достаточно компактной, но при этом достаточной для решения задачи CartPole-v1.


```

▶ # Реализация Replay Buffer и нейронной сети DQN для среды CartPole-v1

from collections import deque, namedtuple

Transition = namedtuple(
    "Transition",
    ("state", "action", "reward", "next_state", "done")
)

class ReplayBuffer:
    def __init__(self, capacity=10000):
        self.memory = deque(maxlen=capacity)

    def push(self, state, action, reward, next_state, done):
        self.memory.append(
            Transition(state, action, reward, next_state, done)
        )

    def sample(self, batch_size):
        transitions = random.sample(self.memory, batch_size)
        return Transition(*zip(*transitions))

    def __len__(self):
        return len(self.memory)

class DQN(nn.Module):
    def __init__(self, n_observations, n_actions):
        super(DQN, self).__init__()

        self.network = nn.Sequential(
            nn.Linear(n_observations, 128),
            nn.ReLU(),
            nn.Linear(128, 128),
            nn.ReLU(),
            nn.Linear(128, n_actions)
        )

    def forward(self, x):
        return self.network(x)

test_dqn_model = DQN(cartpole_n_observations, cartpole_n_actions).to(device)

print("Архитектура DQN:")
print(test_dqn_model)

```

Рисунок 14 — Код реализации Replay Buffer и нейронной сети DQN


```

...  Архитектура DQN:
      DQN(
        (network): Sequential(
          (0): Linear(in_features=4, out_features=128, bias=True)
          (1): ReLU()
          (2): Linear(in_features=128, out_features=128, bias=True)
          (3): ReLU()
          (4): Linear(in_features=128, out_features=2, bias=True)
        )
      )

```

Рисунок 15 — Архитектура нейронной сети DQN

4.11. Реализация агента DQN

На следующем этапе был реализован агент DQN для среды CartPole-v1. Агент включает основную нейронную сеть, целевую нейронную сеть, оптимизатор Adam, буфер воспроизведения опыта и epsilon-greedy стратегию выбора действий. Основная сеть используется для выбора действий и обучения, а целевая сеть применяется для вычисления целевых Q-значений, что повышает устойчивость процесса обучения.

При выборе действия агент использует epsilon-greedy стратегию. С некоторой вероятностью epsilon выбирается случайное действие, что обеспечивает исследование среды. В остальных случаях агент выбирает действие с максимальным Q-значением, предсказанным нейронной сетью. В процессе обучения значение epsilon постепенно уменьшается, что позволяет сначала исследовать среду, а затем чаще использовать найденную стратегию.

Обучение выполняется на мини-пакетах переходов, случайно выбранных из Replay Buffer. Для каждого перехода вычисляется текущее Q-значение и целевое Q-значение. После этого рассчитывается функция потерь MSE, выполняется обратное распространение ошибки и обновление параметров основной сети. Целевая сеть периодически синхронизируется с основной сетью.

```

▶ # Реализация агента DQN для среды CartPole-v1

class DQNAgent:
    def __init__(
        self,
        environment,
        batch_size=128,
        gamma=0.99,
        epsilon_start=1.0,
        epsilon_end=0.05,
        epsilon_decay=0.995,
        learning_rate=1e-3,
        target_update_frequency=10,
        memory_capacity=10000
    ):
        self.env = environment
        self.n_observations = environment.observation_space.shape[0]
        self.n_actions = environment.action_space.n

        self.batch_size = batch_size
        self.gamma = gamma
        self.epsilon = epsilon_start
        self.epsilon_end = epsilon_end
        self.epsilon_decay = epsilon_decay
        self.learning_rate = learning_rate
        self.target_update_frequency = target_update_frequency

        self.policy_net = DQN(self.n_observations, self.n_actions).to(device)
        self.target_net = DQN(self.n_observations, self.n_actions).to(device)
        self.target_net.load_state_dict(self.policy_net.state_dict())
        self.target_net.eval()

        self.optimizer = optim.Adam(self.policy_net.parameters(), lr=self.learning_rate)
        self.memory = ReplayBuffer(capacity=memory_capacity)

        self.episode_rewards = []
        self.episode_steps = []
        self.losses = []

    def choose_action(self, state):
        if np.random.random() < self.epsilon:
            return self.env.action_space.sample()

        state_tensor = torch.tensor(state, dtype=torch.float32, device=device).unsqueeze(0)

        with torch.no_grad():
            q_values = self.policy_net(state_tensor)

        return int(torch.argmax(q_values).item())

    def optimize_model(self):
        if len(self.memory) < self.batch_size:
            return None

        transitions = self.memory.sample(self.batch_size)

        states = torch.tensor(np.array(transitions.state), dtype=torch.float32, device=device)
        actions = torch.tensor(transitions.action, dtype=torch.long, device=device).unsqueeze(1)
        rewards = torch.tensor(transitions.reward, dtype=torch.float32, device=device).unsqueeze(1)
        next_states = torch.tensor(np.array(transitions.next_state), dtype=torch.float32, device=device)
        dones = torch.tensor(transitions.done, dtype=torch.float32, device=device).unsqueeze(1)

        current_q_values = self.policy_net(states).gather(1, actions)

        with torch.no_grad():
            next_q_values = self.target_net(next_states).max(1)[0].unsqueeze(1)
            target_q_values = rewards + self.gamma * next_q_values * (1 - dones)

        loss = F.mse_loss(current_q_values, target_q_values)

        self.optimizer.zero_grad()
        loss.backward()
        self.optimizer.step()

        return loss.item()

    def train(self, episodes=500, max_steps=500):
        start_time = time.time()

        for episode in tqdm(range(episodes), desc="DQN"):
            state, info = self.env.reset(seed=SEED + episode)

            total_reward = 0
            steps = 0
            done = False

            while not done and steps < max_steps:
                action = self.choose_action(state)
                next_state, reward, terminated, truncated, info = self.env.step(action)
                done = terminated or truncated

                self.memory.push(state, action, reward, next_state, done)

                loss = self.optimize_model()
                if loss is not None:
                    self.losses.append(loss)

                state = next_state
                total_reward += reward
                steps += 1

            self.episode_rewards.append(total_reward)
            self.episode_steps.append(steps)

            self.epsilon = max(self.epsilon_end, self.epsilon * self.epsilon_decay)

            if episode % self.target_update_frequency == 0:
                self.target_net.load_state_dict(self.policy_net.state_dict())

        training_time = time.time() - start_time

        return {
            "Алгоритм": "DQN",
            "Количество эпизодов": episodes,
            "Время обучения, сек": training_time,
            "Финальное epsilon": self.epsilon,
            "Средняя награда за последние 50 эпизодов": np.mean(self.episode_rewards[-50:])
        }

print("Класс DQNAgent успешно создан")

```

Класс DQNAgent успешно создан

Рисунок 16 — Код реализации агента DQN и результат создания класса

4.12. Обучение агента DQN

После реализации агента было выполнено обучение алгоритма DQN в среде CartPole-v1. Для обучения использовалось 500 эпизодов, максимальное количество шагов в одном эпизоде было равно 500. В начале обучения агент чаще выбирал случайные действия, что позволяло исследовать среду. По мере обучения значение `epsilon` постепенно уменьшалось, и агент чаще выбирал действия на основе Q-значений, предсказанных нейронной сетью.

По результатам обучения время выполнения составило приблизительно 153,38 секунды. Финальное значение `epsilon` составило 0,081572. Средняя награда за последние 50 эпизодов достигла 216,58. Это показывает, что в процессе обучения агент смог сформировать стратегию, позволяющую достаточно долго удерживать шест в вертикальном положении.

Несмотря на то что обучение DQN может быть менее стабильным по сравнению с табличными методами, данный алгоритм позволяет работать с непрерывным пространством наблюдений, где использование обычной Q-таблицы является неэффективным.

▶ # Обучение агента DQN на среде CartPole-v1

```
dqn_agent = DQNAgent(cartpole_env)

dqn_training_results = dqn_agent.train(
    episodes=500,
    max_steps=500
)

dqn_training_results_df = pd.DataFrame([dqn_training_results])

display(dqn_training_results_df)
```

... DQN: 100% |██████████| 500/500 [02:33<00:00, 3.26it/s]

	Алгоритм	Количество эпизодов	Время обучения, сек	Финальное epsilon	Средняя награда за последние 50 эпизодов
--	----------	---------------------	---------------------	-------------------	--

0	DQN	500	153.379056	0.081572	216.58
---	-----	-----	------------	----------	--------

Рисунок 17 — Результаты обучения агента DQN

4.13. Оценка качества обученного агента DQN

После завершения обучения была выполнена оценка качества агента DQN на 100 тестовых эпизодах. На этапе тестирования значение `epsilon` было временно установлено равным 0, что позволило оценивать поведение агента без случайного исследования среды. Таким образом, агент выбирал действия только на основе Q-значений, предсказанных обученной нейронной сетью.

Результаты тестирования показали, что средняя награда составила 500, среднее количество шагов также составило 500. Максимальная и минимальная награда на тестовых эпизодах также были равны 500. Это означает, что агент успешно удерживал шест в вертикальном положении до максимального ограничения эпизода во всех тестовых запусках.

Полученный результат показывает, что обученный агент DQN успешно решил задачу CartPole-v1. В отличие от табличных алгоритмов, DQN использует нейронную сеть для аппроксимации Q-функции, что делает данный подход применимым к задачам с непрерывным пространством наблюдений.

▶ # Оценка качества обученного агента DQN на тестовых эпизодах

```
def evaluate_dqn_agent(environment, agent, episodes=100, max_steps=500, seed=42):
    total_rewards = []
    total_steps = []

    old_epsilon = agent.epsilon
    agent.epsilon = 0.0

    for episode in range(episodes):
        state, info = environment.reset(seed=seed + episode)

        episode_reward = 0
        steps = 0
        done = False

        while not done and steps < max_steps:
            action = agent.choose_action(state)
            next_state, reward, terminated, truncated, info = environment.step(action)
            done = terminated or truncated

            episode_reward += reward
            steps += 1
            state = next_state

        total_rewards.append(episode_reward)
        total_steps.append(steps)

    agent.epsilon = old_epsilon

    results = {
        "Алгоритм": "DQN",
        "Количество тестовых эпизодов": episodes,
        "Средняя награда": np.mean(total_rewards),
        "Среднее количество шагов": np.mean(total_steps),
        "Максимальная награда": np.max(total_rewards),
        "Минимальная награда": np.min(total_rewards)
    }

    return results

dqn_eval_results = evaluate_dqn_agent(
    environment=cartpole_env,
    agent=dqn_agent,
    episodes=100,
    max_steps=500,
    seed=SEED
)

dqn_eval_results_df = pd.DataFrame([dqn_eval_results])

display(dqn_eval_results_df)
```

Рисунок 18 — Код оценки качества обученного агента DQN

	Алгоритм	Количество тестовых эпизодов	Средняя награда	Среднее количество шагов	Максимальная награда	Минимальная награда
0	DQN	100	500.0	500.0	500.0	500.0

Рисунок 19 — Результат оценки качества обученного агента DQN

4.14. Визуализация процесса обучения агента DQN

Для анализа динамики обучения агента DQN был построен график изменения суммарной награды по эпизодам. На графике отображаются значения награды за отдельные эпизоды, а также скользящее среднее с окном 20 эпизодов. Использование скользящего среднего позволяет более наглядно оценить общую тенденцию обучения, так как отдельные значения награды могут заметно колебаться.

По графику видно, что процесс обучения DQN является менее стабильным, чем обучение табличных алгоритмов. Это связано с тем, что нейронная сеть обучается приближенно, а поведение агента зависит от текущих параметров сети, случайного исследования среды и содержимого Replay Buffer. В разные моменты обучения агент может показывать как высокие, так и низкие значения награды.

Несмотря на колебания во время обучения, итоговая оценка качества показала, что обученный агент успешно решает задачу CartPole-v1. Это подтверждает, что DQN смог сформировать эффективную стратегию управления тележкой и удержания шеста в вертикальном положении.

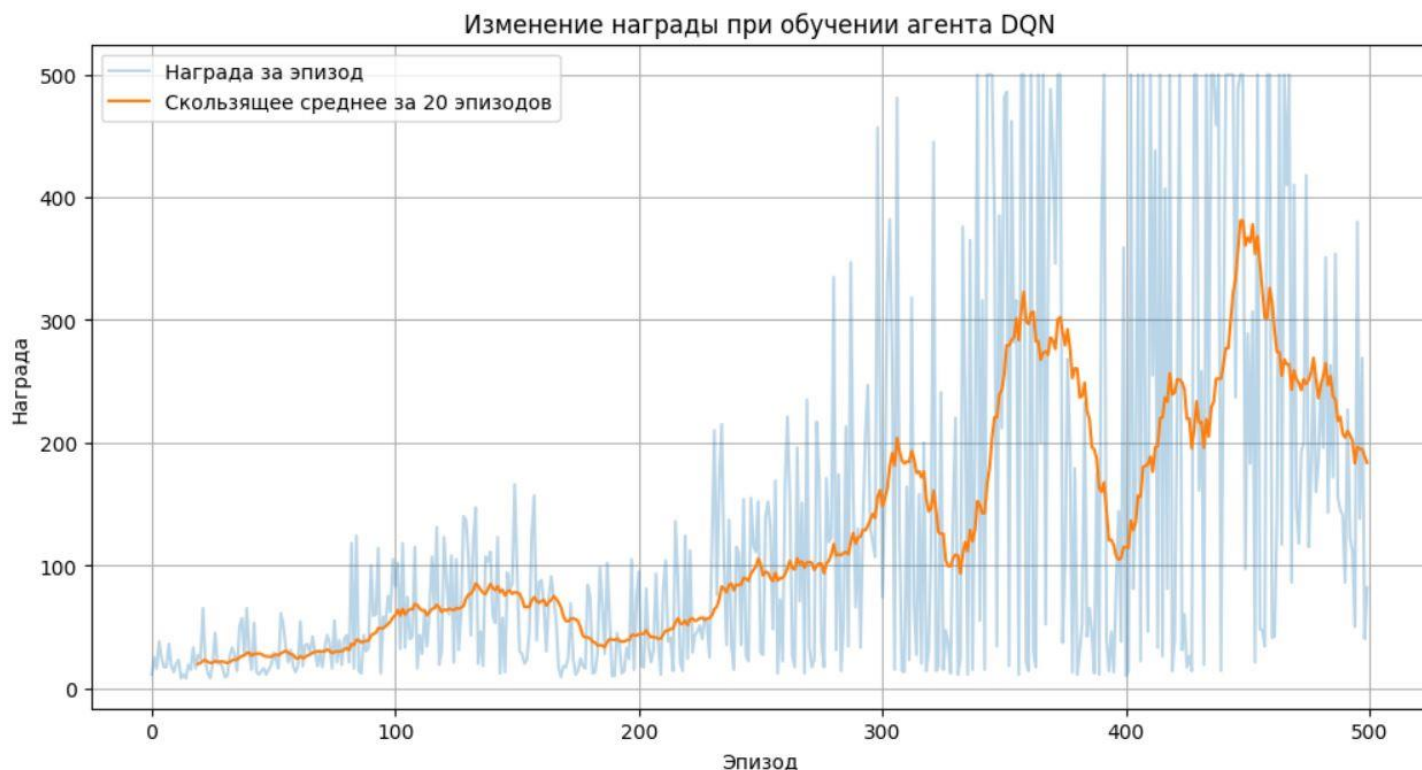


Рисунок 20 — График изменения награды при обучении агента DQN

4.15. Итоговые результаты практического этапа

В завершение практического этапа была сформирована итоговая таблица, объединяющая результаты табличных методов и метода глубокого обучения с подкреплением. В таблицу были включены алгоритмы Policy Iteration, Q-Learning и DQN. Такой выбор позволяет сопоставить классический метод динамического программирования, метод временных различий и метод глубокого обучения с подкреплением.

Для среды Taxi-v4 оба табличных алгоритма успешно решили задачу и достигли доли успешных эпизодов 1,0. Policy Iteration показал среднюю награду 7,34 и среднее количество шагов 13,66, а также быстро завершил обучение. Однако данный метод требует известной модели переходов среды. Q-Learning достиг такого же качества по средней награде и количеству шагов и при этом обучался непосредственно на основе взаимодействия агента со средой.

Для среды CartPole-v1 алгоритм DQN показал среднюю награду 500 и среднее количество шагов 500 на тестовых эпизодах. Это означает, что агент успешно удерживал шест до максимального ограничения эпизода. Полученные результаты подтверждают, что табличные методы эффективны для дискретных сред, а DQN позволяет решать задачи с непрерывным пространством наблюдений с использованием нейронной сети.

	Группа методов	Среда	Алгоритм	Количество эпизодов	Время обучения, сек	Средняя награда	Среднее количество шагов	Доля успешных эпизодов
0	Табличные методы	Taxi-v4	Policy Iteration	-	1.618330	7.34	13.66	1.0
1	Табличные методы	Taxi-v4	Q-Learning	8000	10.804644	7.34	13.66	1.0
2	Глубокое обучение с подкреплением	CartPole-v1	DQN	500	153.379056	500.00	500.00	-

Рисунок 21 — Итоговая таблица результатов практического этапа

5. Заключение

В ходе выполнения домашнего задания была рассмотрена задача обучения с подкреплением и выполнен сравнительный анализ табличных методов и метода глубокого обучения с подкреплением. В теоретической части были рассмотрены основные понятия RL, включая агента, среду, состояние, действие, вознаграждение, стратегию, функцию ценности и Q-функцию. Также были описаны особенности алгоритмов Policy Iteration, SARSA, Q-Learning, Double Q-Learning и Deep Q-Network.

В практической части были реализованы алгоритмы Policy Iteration, SARSA, Q-Learning и Double Q-Learning для среды Taxi-v4. Полученные результаты показали, что все табличные методы успешно решили задачу управления такси и достигли доли успешных эпизодов 1,0. Алгоритм Policy Iteration показал высокое качество стратегии и быстро завершил обучение, однако его применение возможно только при наличии известной модели переходов среды. Алгоритмы Q-Learning и Double Q-Learning показали лучшие значения средней награды среди методов временных различий.

Для исследования глубокого обучения с подкреплением был реализован алгоритм DQN в среде CartPole-v1. В данной задаче состояние задается непрерывным вектором признаков, поэтому вместо Q-таблицы использовалась нейронная сеть. По результатам тестирования агент DQN достиг средней награды 500 на 100 тестовых эпизодах, что соответствует успешному решению задачи.

Таким образом, практические эксперименты подтвердили, что табличные методы эффективны для сред с конечным дискретным пространством состояний и действий, а методы глубокого обучения с подкреплением позволяют решать задачи с непрерывными пространствами наблюдений. Полученные результаты соответствуют теоретическим особенностям рассмотренных алгоритмов.

6. Список использованных источников

1. Sutton R. S., Barto A. G. Reinforcement Learning: An Introduction. 2nd ed. Cambridge, MA: MIT Press, 2018. URL: <https://incompleteideas.net/book/the-book-2nd.html>
2. Watkins C. J. C. H., Dayan P. Q-Learning // Machine Learning. 1992. Vol. 8. P. 279–292. URL: <https://link.springer.com/article/10.1007/BF00992698>
3. Van Hasselt H. Double Q-learning // Advances in Neural Information Processing Systems. 2010. Vol. 23. URL: <https://papers.nips.cc/paper/3964-double-q-learning>
4. Mnih V., Kavukcuoglu K., Silver D., Rusu A. A., Veness J., Bellemare M. G., Graves A., Riedmiller M., Fidjeland A. K., Ostrovski G., Petersen S., Beattie C., Sadik A., Antonoglou I., King H., Kumaran D., Wierstra D., Legg S., Hassabis D. Human-level control through deep reinforcement learning // Nature. 2015. Vol. 518. P. 529–533. URL: <https://www.nature.com/articles/nature14236>
5. Gymnasium Documentation. Taxi environment. Farama Foundation. URL: https://gymnasium.farama.org/environments/toy_text/taxi/
6. Gymnasium Documentation. CartPole environment. Farama Foundation. URL: https://gymnasium.farama.org/environments/classic_control/cart_pole/
7. PyTorch Documentation. Neural Networks. URL: https://docs.pytorch.org/tutorials/beginner/basics/buildmodel_tutorial.html
8. PyTorch Documentation. torch.optim. URL: <https://docs.pytorch.org/docs/stable/optim.html>
9. Gymnasium Documentation. Basic Usage. URL: https://gymnasium.farama.org/introduction/basic_usage/
10. Matplotlib Documentation. Pyplot tutorial. URL: <https://matplotlib.org/stable/tutorials/pyplot.html>.